

ECE 759

High Performance Computing for Engineering Applications

Assignment 2

Submitted by: Zhengyang Wan

My GitHub link: <https://github.com/BillWan-zzzzyy/repo759/tree/main/HW02>

1.

- (a) Implement the `scan` function in a file called `scan.cpp` with signature defined as in `scan.h`. You should write an inclusive scan on your own and not use any library scan functions.

Answer:

- (b) Write a file `task1.cpp` with a `main` function which (in this order)
- i) Creates an array of `n` random `float` numbers between -1.0 and 1.0. `n` should be read as the first command line argument as below.
 - ii) Scans the array using your `scan` function.
 - iii) Prints out the time taken by your `scan` function in *milliseconds*.
 - iv) Prints the first element of the output scanned array.
 - v) Prints the last element of the output scanned array.
 - vi) Deallocates memory when necessary.

Compile command: `g++ scan.cpp task1.cpp -Wall -O3 -std=c++17 -o task1`

Run command: `./task1 n`

Answer:

- (c) On an *Euler* compute node (using a Slurm Job), run `task1` for each value $n = 2^{10}, 2^{11}, \dots, 2^{30}$ and generate a plot `task1.pdf` (with axis labels) which plots the time taken by your algorithm as a function of `n`. This is called a scaling analysis.

Answer:

2.

Convolutions appear very prominently in image processing and in other fields like the numerical solution of partial differential equations, machine learning, etc.

- (a) Implement the `convolve` function in a file called `convolution.cpp` with signature as provided in `convolution.h`. The operation that needs to be implemented is as follows:

$$g[x, y] = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} \omega[i, j] f\left[x + i - \frac{m-1}{2}, y + j - \frac{m-1}{2}\right],$$

where $x, y = 0, \dots, n-1$; f is the original image, ω is the mask, and g is the result of the convolution; m is the dimension of the square matrix ω , where $m \geq 1$ is assumed to be an odd number. We assume that $f[i, j] = 1$ if one of the following two conditions is not satisfied, and $f[i, j] = 0$ if neither of the following two conditions is satisfied:

$$\begin{cases} 0 \leq i < n, \\ 0 \leq j < n. \end{cases}$$

In other words, we will pad zeros for corners, and pad ones for edges (excluding the corners).

Example (uses $n = 4$ and $m = 3$):

$$f = \begin{bmatrix} 1 & 3 & 4 & 8 \\ 6 & 5 & 2 & 4 \\ 3 & 4 & 6 & 8 \\ 1 & 4 & 5 & 2 \end{bmatrix}, \quad \omega = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad g = \begin{bmatrix} 3 & 10 & 10 & 10 \\ 10 & 12 & 14 & 11 \\ 9 & 7 & 14 & 14 \\ 5 & 11 & 14 & 4 \end{bmatrix}$$

Answer:

- (b) Write a file `task2.cpp` with a `main` function which (in this order)
- Creates an `n×n image` matrix (stored in 1D in row-major order) of random `float` numbers between -10.0 and 10.0. The value of `n` should be read as the first command line argument.
 - Creates an `m×m mask` matrix (stored in 1D in row-major order) of random `float` numbers between -1.0 and 1.0. The value of `m` should be read as the second command line argument.
 - Applies the `mask` to `image` using your `convolve` function.
 - Prints out the time taken by your `convolve` function in *milliseconds*.
 - Prints the first element of the resulting convolved array.

- vi) Prints the last element of the resulting convolved array.
- vii) Deallocates memory when necessary via the `delete` function.

Compile command: `g++ convolution.cpp task2.cpp -Wall -O3 -std=c++17 -o task2`
 Run command: `./task2 n m`

Answer:

3.

Implement in a file called `matmul.cpp` the four functions with signatures and descriptions as in `matmul.h` to produce the matrix product $C = AB$. For all of the cases, the array `C` that stores the matrix C should be reported in row-major order.

- (a) `mmul1` should have three `for` loops: the outer loop sweeps index `i` through the rows of `C`, the middle loop sweeps index `j` through the columns of `C`, and the innermost loop sweeps index `k` through; i.e., to carry out, the dot product of the i^{th} row `A` with the j^{th} column of `B`. Inside the innermost loop, you should have a single line of code which increments C_{ij} . Assume that `A` and `B` are 1D arrays storing the matrices in row-major order.

Answer:

- (b) `mmul2` should also have three `for` loops, but the two innermost loops should be swapped relative to `mmul1` (`(i,j,k)` becomes `(i,k,j)`).

Answer:

- (c) `mmul3` should also have three `for` loops, but the outermost loop in `mmul1` should become the innermost loop in `mmul3` (`(i,j,k)` becomes `(j,k,i)`).

Answer:

- (d) `mmul4` should have the `for` loops ordered as in `mmul1`, but this time around `A` and `B` are stored as `std::vector<double>`.

Answer:

- (e) Write a program `task3.cpp` that generates square matrices `A` and `B` of dimension at least 1000×1000 stored in row-major order, computes $C = AB$ using each of your functions, and prints the number of rows of your input matrices, and for each `mmul` function in ascending order, prints the amount of time taken in *milliseconds* and the last element of the resulting `C` (nine values, one per line).

Compile command: `g++ task3.cpp matmul.cpp -Wall -O3 -std=c++17 -o task3`
Run command: `./task3`

Answer:

- (f) In a couple sentences, explain the difference that you see in the times for `mmul1`, `mmul2` and `mmul3` *when running on an Euler compute node*. What would explain the performance results you report? Be as specific as possible. Also comment on the difference or similarity you see between `mmul1` and `mmul4`.

Answer: